

Real-Time Chat Application

Internship Project Report · Elevate Labs · 2-Week Phase

Project	Real-Time Chat App with Socket.IO
Intern	Hrithika S.
Domain	Full-Stack Web Development
Stack	Node.js · Express · Socket.IO · MongoDB · React · Tailwind CSS
Duration	2 Weeks

1. Introduction

Real-time communication has become fundamental to modern web applications. This project implements a production-ready chat platform that enables instant messaging between users through persistent WebSocket connections. The application supports group rooms and private direct messages, provides live typing indicators, tracks online presence, and stores complete message history in a persistent database. It is built entirely on the MERN-adjacent stack (Node.js, Express, MongoDB, React) with Socket.IO powering the bidirectional real-time layer, and Tailwind CSS delivering a polished dark-mode interface.

2. Abstract

The ChatApp platform provides a secure, scalable real-time messaging experience. On the backend, an Express.js server handles REST API requests for authentication and data retrieval, while Socket.IO manages persistent WebSocket connections for live events. User credentials are hashed with bcrypt and protected by JSON Web Tokens (JWT). MongoDB with Mongoose stores users, rooms, and messages. The React frontend manages state through two context providers—AuthContext and SocketContext—and renders a multi-panel chat UI styled with Tailwind CSS utility classes.

3. Tools Used

Technology	Version	Purpose
Node.js	v20 LTS	JavaScript runtime for the backend server
Express.js	4.18	REST API framework and middleware chain
Socket.IO	4.6	Bidirectional real-time event communication
MongoDB	7.x	NoSQL document database for persistence
Mongoose	8.x	ODM for schema definition and queries
JSON Web Token	9.x	Stateless authentication tokens
bcryptjs	2.4	Password hashing with salt rounds
React	18.2	Frontend UI library with hooks
React Router	v6	Client-side page routing (SPA)
Tailwind CSS	3.x	Utility-first CSS framework
Axios	1.6	HTTP client for API requests

date-fns	3.x	Lightweight date formatting library
----------	-----	-------------------------------------

4. Steps Involved in Building the Project

Step 1 — Project Architecture & Setup

Initialised two separate Node.js projects: a backend package with Express, Socket.IO, Mongoose, bcryptjs, jsonwebtoken, and express-validator; and a React frontend bootstrapped with Create React App. Defined the folder structure separating models, routes, and middleware on the server side, and pages, components, context, and utilities on the client side.

Step 2 — Database Modelling (MongoDB / Mongoose)

Designed three Mongoose schemas: User (username, email, hashed password, online status, lastSeen), Room (name, type group/dm, members array, admins, lastMessage reference, isPrivate flag), and Message (room reference, sender reference, content, type, readBy array, edited flag, soft-delete via deletedAt). Password hashing is performed in a pre-save hook.

Step 3 — JWT Authentication (REST API)

Built `/api/auth/register` and `/api/auth/login` endpoints that validate input with express-validator, hash passwords with bcrypt (12 salt rounds), and issue 7-day JWT tokens. An authenticate middleware verifies tokens on every protected REST route. A parallel authenticateSocket middleware authenticates every Socket.IO connection via the handshake `auth.token` field before the connection is established.

Step 4 — REST API Endpoints

Implemented complete CRUD routes for Users (search, profile update), Rooms (list, create, join, leave, add member), and Messages (paginated fetch, soft delete, edit). All routes are protected and return JSON; error responses include descriptive messages and appropriate HTTP status codes.

Step 5 — Socket.IO Real-Time Layer

On connection, the socket handler marks the user online, joins all their existing rooms, and broadcasts the online event. Handled events include: `message:send` (saves to MongoDB, broadcasts to room), `dm:send` (finds or creates a DM room, delivers to both parties), `typing:start` / `typing:stop` (relayed to room peers), `message:read` (bulk update `readBy`), and `disconnect` (marks offline, broadcasts status change). An in-memory Map tracks `userId` → `socketId` for targeted DM delivery.

Step 6 — React Frontend (Context + Components)

AuthContext manages token storage and user state; SocketContext initialises the Socket.IO client after login and tears it down on logout. ChatPage is the top-level layout that fetches rooms and listens for real-time updates. Sidebar lists rooms with last-message previews and online dots. ChatWindow fetches message history with pagination, scrolls to bottom, and sends messages via socket. MessageBubble renders own/other bubbles with timestamps and a delete action. TypingIndicator shows animated dots while peers type.

Step 7 — Group Rooms & Direct Messages

CreateRoomModal allows creating named group rooms (with optional member invites and privacy toggle) or starting a DM by searching users. DiscoverRoomsModal lists all public rooms with member counts and a one-click Join button. DM rooms are keyed by the sorted pair of user IDs so they are reused across sessions.

Step 8 — UI Polish & Testing

Applied a consistent dark-mode design using Tailwind utilities: slate-700/800/900 backgrounds, blue-600 accents, gradient avatars colour-coded by username initial. Added CSS keyframe animations for message entry, typing dots, and online-status pulse. Tested multi-user scenarios by opening the app in two separate browser windows and verifying real-time delivery, typing indicators, online/offline toggles, and DM routing.

5. Conclusion

The Real-Time Chat Application successfully demonstrates the integration of REST APIs and persistent WebSocket connections into a cohesive full-stack product. The project reinforced key concepts including event-driven architecture, JWT-based stateless authentication, MongoDB document design, and React state management through context. Challenges such as synchronising socket events with REST-managed state and handling DM room creation idempotently were solved cleanly. The codebase is modular and production-ready, with clearly separated concerns, environment-variable configuration, and a documented REST + Socket.IO API. This project serves as a strong portfolio piece covering both backend engineering and modern frontend development.